Reinforcement Learning Part II

Arash Marioriyad

9 Tir 1405

CE 417-Artificial Intelligence
Sharif Univ. of Tech.

Some of Slides have been adopted from Berkeley CS188 & Berkeley CS285 & Sharif UT CE417

Reinforcement Learning

- Still assume a Markov decision process (MDP):
  - A set of states s ∈ S
  - A set of actions (per state) A
  - A model T(s,a,s')
  - A reward function R(s,a,s')
- Still looking for a policy π(s)

- New twist: don't know T or R
  - I.e. we don't know which states are good or what the actions do
  - Must try out actions and states to learn
    - Q1: How to learn from things tried? (today, Passive Reinforcement Learning)
    - Q2: What to decide to try? (Thursday, Active Reinforcement Learning)

Trajectory and Return/Utility

- An agent can move along states by using actions according to a policy π:

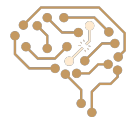
$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} \dots$$

- So the Agent receives a reward per each move ; i.e. $R(s_0, a_0, s_1) = r_0$

$$s_0, a_0, r_0, s_1, a_1, r_1, s_2, a_2, r_2, \dots$$

- The goal of the agent is to choose a policy π that maximizes the (discounted) sum of rewards along a path (trajectory); we name below sum utility or return

$$U([s_0, a_0, s_1, a_1, s_2, \dots]) = R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \gamma^2 R(s_2, a_2, s_3) + \dots$$



Value

- $\mathbf{V^{\pi}(s)}$ = expected return starting from s and acting based on policy π
- $\mathbf{V^{*}(s)}$ = expected return starting from s and acting optimally (π^{*})
- $\mathbf{Q^{\pi}(s, a)}$ = expected return starting from s and choosing action a, then acting based on policy π
- $\mathbf{Q^{*}(s, a)}$ = expected return starting from s and choosing action a, then acting optimally (π^{*})
- Why expected? Due to the randomness, when acting based on π or π^{*}, we may experience different paths, so we must make an expectation

Bellman Equations

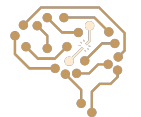
- Recursive definition of value:

$$V^*(s) = \max_a Q^*(s,a)$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma\, V^*(s')\right]$$

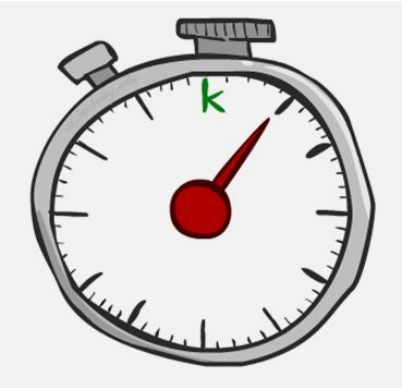
$$V^*(s) = \max_a \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma V^*(s')\right]$$

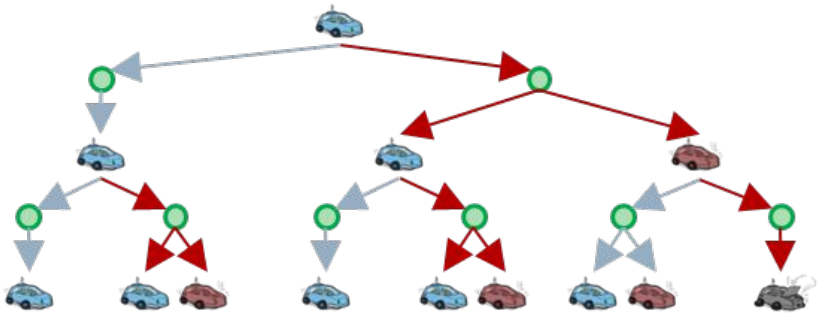
$$V^\pi(s) = \sum_{s'} T(s,\pi(s),s')\left[R(s,\pi(s),s') + \gamma V^\pi(s')\right]$$

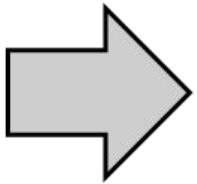


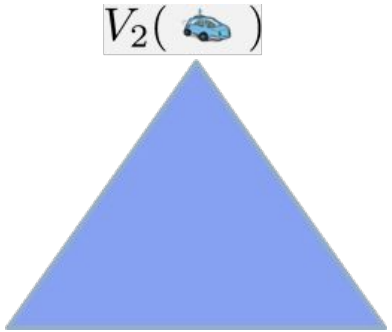
Time-Limited Values

- Define $\mathbf{V_k(s)}$ to be the value of s if the game ends in k more time steps



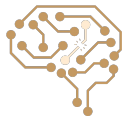




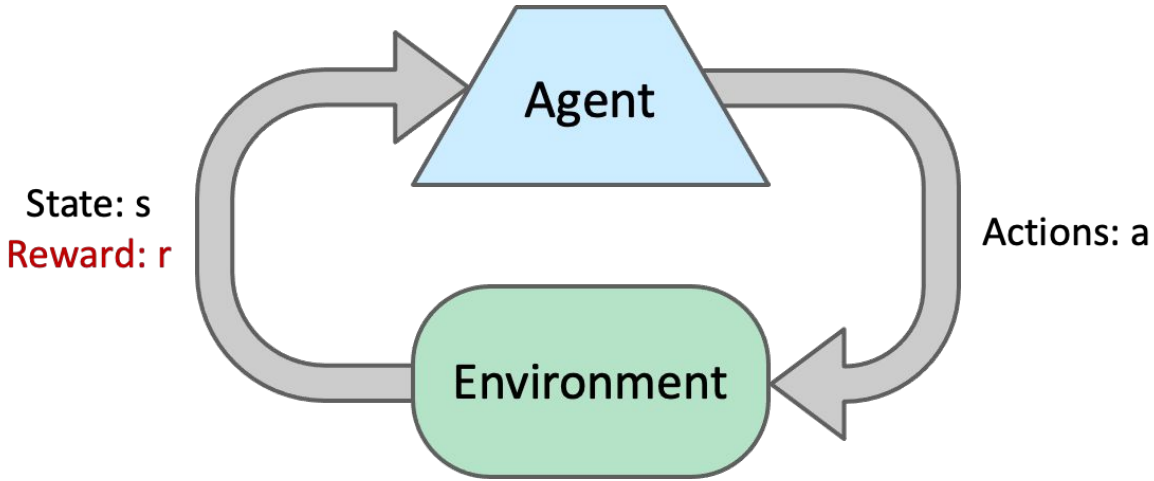


New Issues, in Reinforcement learning

- What if the MDP is initially unknown? Lots of things change!
  - **Exploration**: you have to try unknown actions to get information
  - **Exploitation**: eventually, you have to use what you know
  - **Regret**: early on, you inevitably “make mistakes” and lose reward
  - **Sampling**: you may need to repeat many times to get good estimates
  - **Generalization**: what you learn in one state may apply to others too



Reinforcement Learning (RL)



- Receive **feedback** in the form of reward and next state
- Must (learn to) act so as to **maximize expected rewards**
- All learning is based on **observed samples** of environment!

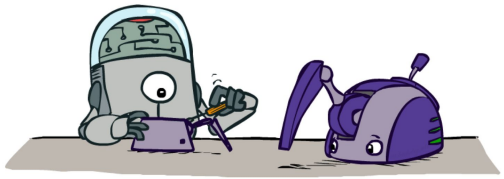
Passive RL vs Active RL

- Passive RL – how to learn to act from data
  - Model-based RL
    - Note: ~equally important as model-free RL, simpler conceptually, hence will take less time / slides
  - Model-free RL
    - Sample-based policy evaluation for Value learning (“monte carlo value estimates”)
    - Temporal Difference Value learning (“TD learning”)
    - Temporal Difference Q-Value learning (“Q learning”)
- Active RL – how to act to collect data
  - i.e. Exploration (vs. Exploitation)
- Scaling up RL
  - Approximate Q learning

Model-Based RL

- Model-Based Idea:
  - Learn an approximate model based on experiences
  - Solve for values as if the learned model were correct

- Step 1: Learn empirical MDP model
  - Count outcomes s' for each s, a
  - Normalize to give an estimate of $\hat{T}(s, a, s')$
  - Discover each $\hat{R}(s, a, s')$ when we experience (s, a, s')

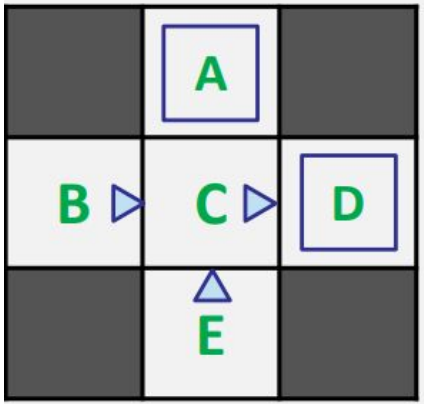


- Step 2: Solve the learned MDP
  - For example, use value iteration, as before

- Step 3: Run the learned policy
  - If happy with result, all done
  - If not happy, add the data (s, a, r, s') to data set and go back to Step 1

Example: Model-Based Learning

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

$T(s,a,s')$

T(B, east, C) = 1.00
P(C, east, D) = 0.75
P(C, east, A) = 0.25
...

$R(s,a,s')$

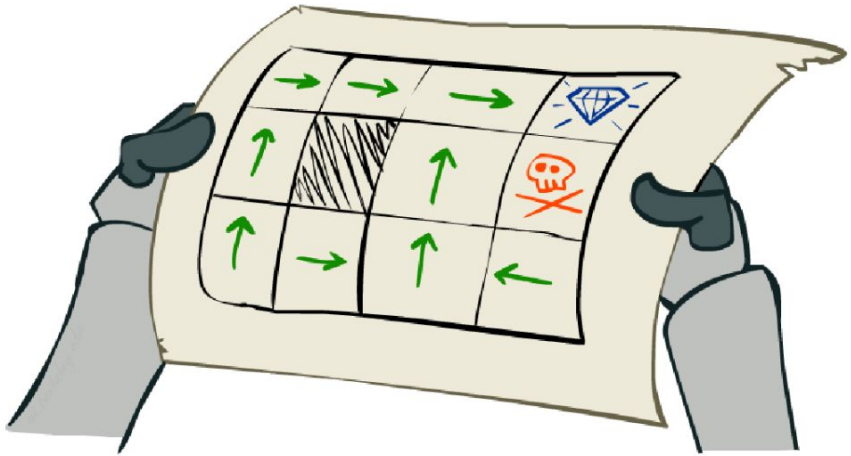
R(B, east, C) = -1
R(C, east, D) = -1
R(D, exit, x) = +10
...

Model-Free Learning

- Direct Evaluation
- Temporal Difference
- Q-learning
- SARSA

Policy Evaluation with Unknown Model: Problem Setting

- Simplified task: policy evaluation
  - Input: a fixed policy $\pi(s)$
  - You don't know the transitions T(s,a,s')
  - You don't know the rewards R(s,a,s')
  - Goal: learn the state values



- In this case:
  - Learner is "along for the ride"
  - No choice about what actions to take
  - Just execute the policy and learn from experience
  - This is NOT offline planning! You actually take actions in the world.

Recall: Policy Evaluation

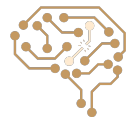
- When model (transition probability and rewards) is known:

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

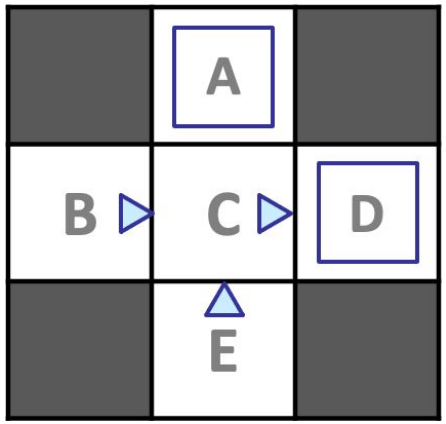
Policy Evaluation: Direct Evaluation From Samples

- Goal: Compute values for each state under π
- Idea: Average together observed sample values
  - Act according to π
  - Every time you visit a state, write down what the sum of discounted rewards turned out to be
  - Average those samples
- This is called direct evaluation



Example: Direct Evaluation From Samples

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

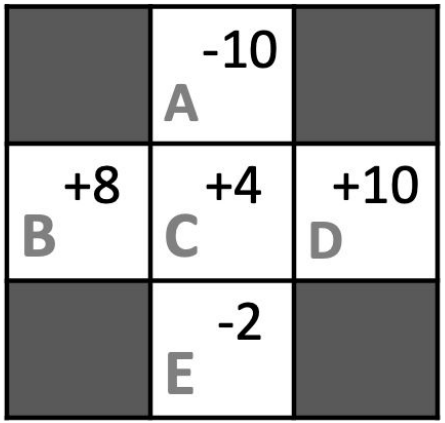
Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

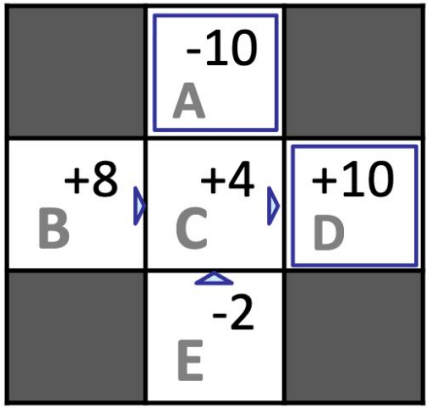
Output Values



Problems with Direct Evaluation

- What's good about direct evaluation?
  - It's easy to understand
  - It doesn't require any knowledge of T, R
  - It eventually computes the correct average values, using just sample transitions
- What bad about it?
  - It wastes information about state connections
  - Each state must be learned separately
  - So, it takes a long time to learn

Output Values



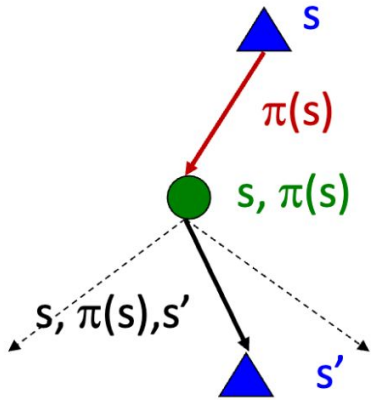
If B and E both go to C under this policy, how can their values be different?

Why Not Use Bellman Updates?

- Simplified Bellman updates calculate V for a fixed policy:
  - Each round, replace V with a one-step-look-ahead layer over V

$$V_0^{\pi}(s) = 0$$

$$V_{k+1}^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^{\pi}(s')]$$



  - This approach fully exploited the connections between the states
  - Unfortunately, we need T and R to do it!

- Key question: how can we do this update to V without knowing T and R?
  - In other words, how to we take a weighted average without knowing the weights?

Sample-Based Bellman Updates?

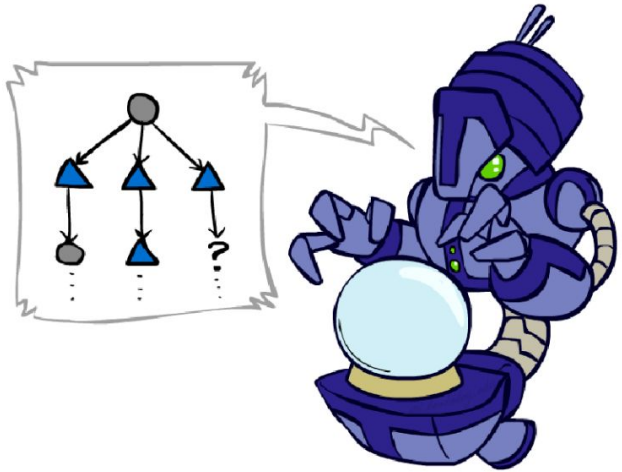
- We want to improve our estimate of V by computing these averages:

$$V_{k+1}^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^{\pi}(s')]$$

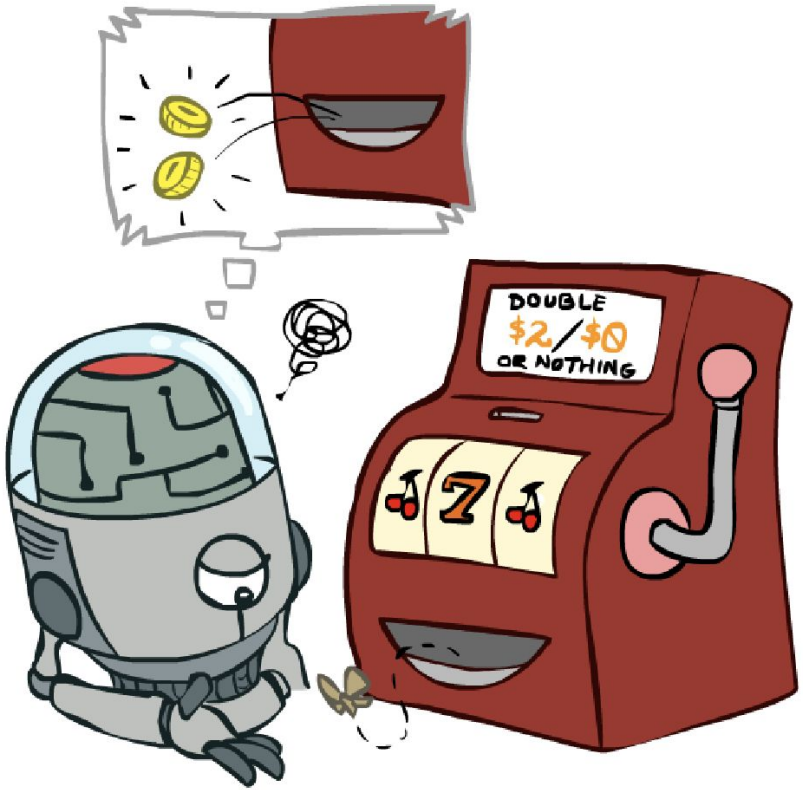
- Idea: Take samples of outcomes s' (by doing the action!) and average

$$sample_1 = R(s, \pi(s), s'_1) + \gamma V_k^{\pi}(s'_1)$$
$$sample_2 = R(s, \pi(s), s'_2) + \gamma V_k^{\pi}(s'_2)$$
$$\ldots$$
$$sample_n = R(s, \pi(s), s'_n) + \gamma V_k^{\pi}(s'_n)$$

$$V_{k+1}^{\pi}(s) \leftarrow \frac{1}{n} \sum_i sample_i$$

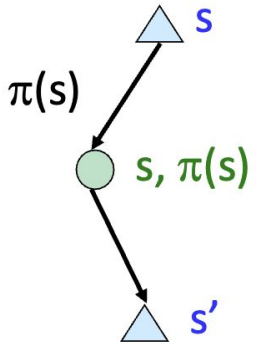


Temporal Difference Learning



Temporal Difference Learning

- Big idea: learn from every experience!
  - Update V(s) each time we experience a transition (s, a, s', r)
  - Likely outcomes s' will contribute updates more often



- Temporal difference learning of values
  - Policy still fixed, still doing evaluation!
  - Move values toward value of whatever successor occurs: running average

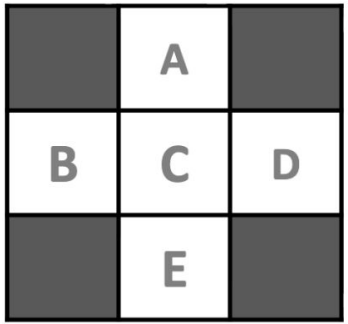
Sample of V(s): $sample = R(s, \pi(s), s') + \gamma V^{\pi}(s')$

Update to V(s): $V^{\pi}(s) \leftarrow (1-\alpha)V^{\pi}(s) + (\alpha)sample$

Same update: $V^{\pi}(s) \leftarrow V^{\pi}(s) + \alpha(sample - V^{\pi}(s))$

Example: Temporal Difference Learning

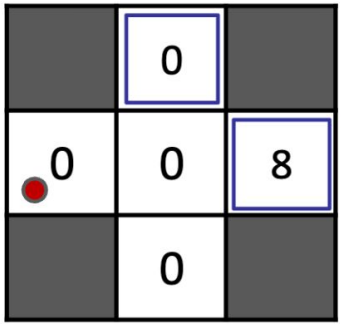
States



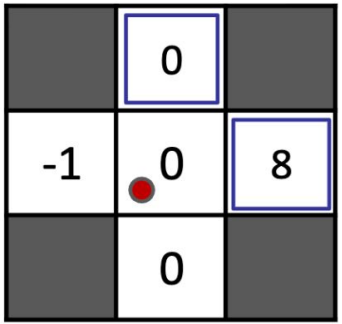
Assume: $\gamma = 1$, $\alpha = 1/2$

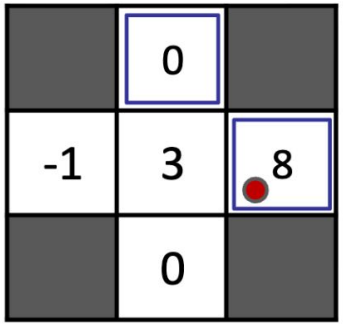
Observed Transitions

B, east, C, -2



C, east, D, -2





$$V^{\pi}(s) \leftarrow (1-\alpha)V^{\pi}(s) + \alpha\left[R(s,\pi(s),s') + \gamma V^{\pi}(s')\right]$$

Problems with TD Value Learning

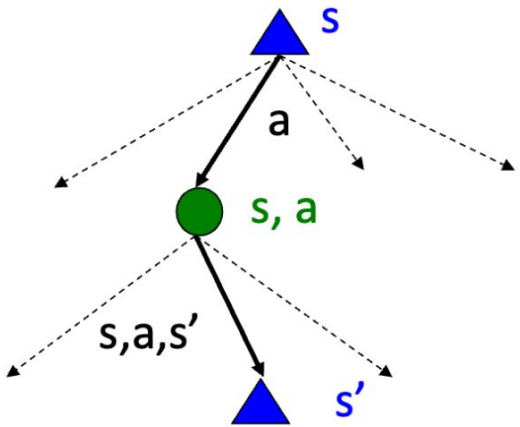
- TD value leaning is a model-free way to do policy evaluation, mimicking Bellman updates with running sample averages
- However, if we want to turn values into an optimal policy, we're sunk:

$$\pi(s) = \arg\max_a Q(s,a)$$

$$Q(s,a) = \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V(s') \right]$$

- Idea: learn Q-values, not values

  Makes action selection model-free too!



Recall: Q-Value Iteration

- Value iteration: find successive (depth-limited) values
  - Start with $V_0(s) = 0$, which we know is right
  - Given V_k, calculate the depth k+1 values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

- But Q-values are more useful, so compute them instead
  - Start with $Q_0(s,a) = 0$, which we know is right
  - Given Q_k, calculate the depth k+1 q-values for all q-states:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$$

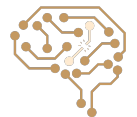
Q-Learning

- Learn Q(s,a) values as you go
  - Receive a sample (s,a,s',r)
  - Consider your old estimate: $Q(s,a)$
  - Consider your new sample estimate:

$$sample = R(s,a,s') + \gamma \max_{a'} Q(s',a')$$

  - Incorporate the new estimate into a running average:

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + (\alpha)\,[sample]$$



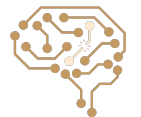
Off-policy RL vs. On-policy RL

- Q-learning (off-policy RL):

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

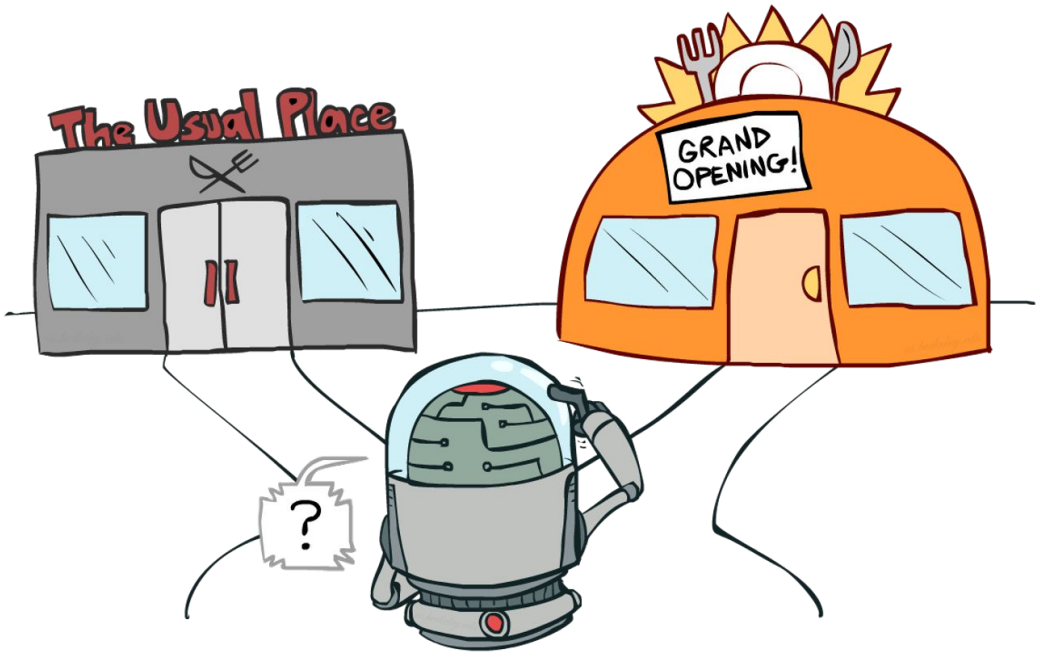
- SARSA (on-policy RL):

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$



Active Reinforcement Learning: How to Explore?

- Exploration vs. Exploitation



How to Explore?

$$a_t = \begin{cases} \text{random action}, & \text{with probability } \varepsilon \\ \arg\max_a Q(s_t, a), & \text{with probability } 1-\varepsilon \end{cases}$$

- Several schemes for forcing exploration
  - Simplest: random actions (ε-greedy)
    - Every time step, flip a coin
    - With (small) probability ε, act randomly
    - With (large) probability 1-ε, act on current policy
  - Problems with random actions?
    - You do eventually explore the space, but keep thrashing around once learning is done
    - One solution: lower ε over time
    - Another solution: exploration functions

Exploration Functions

- When to explore?
  - Random actions: explore a fixed amount
  - Better idea: explore areas whose badness is not (yet) established, eventually stop exploring

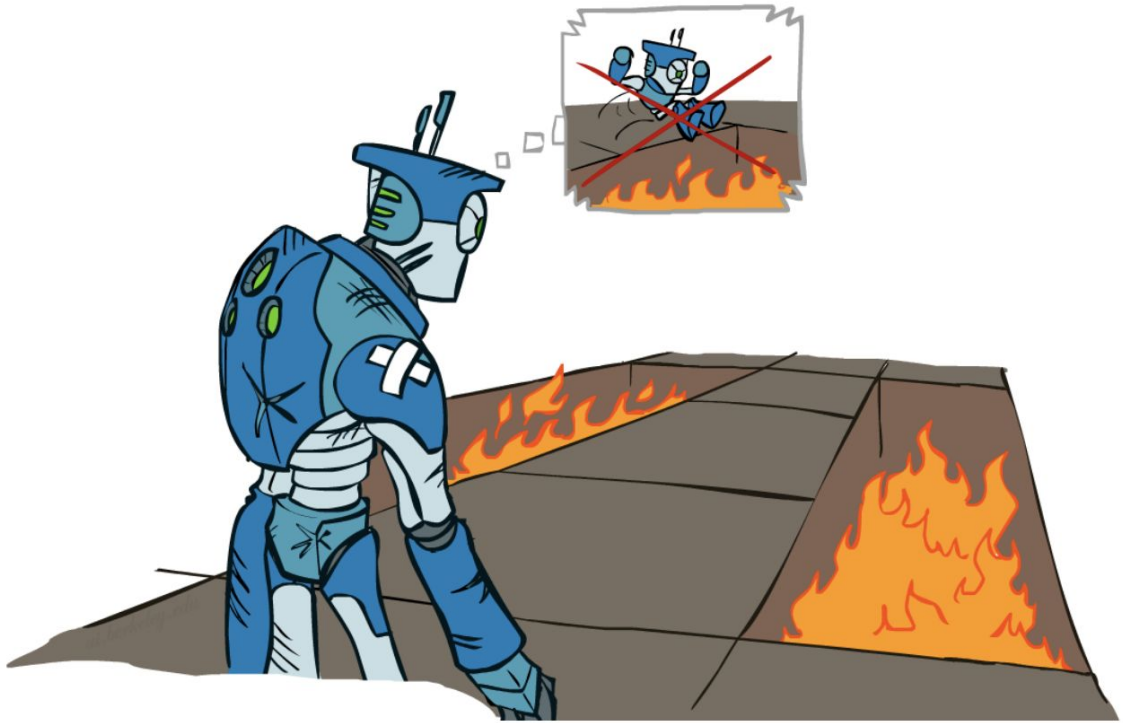
- Exploration function
  - Takes a value estimate u and a visit count n, and returns an optimistic utility, e.g. $f(u, n) = u + k/n$

Regular Q-Update: $Q(s,a) \leftarrow_\alpha R(s,a,s') + \gamma \max_{a'} Q(s',a')$

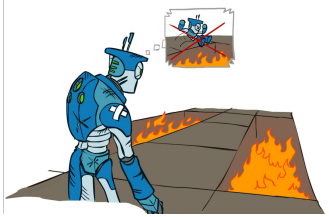
Modified Q-Update: $Q(s,a) \leftarrow_\alpha R(s,a,s') + \gamma \max_{a'} f(Q(s',a'), N(s',a'))$

  - Note: this propagates the "bonus" back to states that lead to unknown states as well!

How can we evaluate RL Methods?



Regret

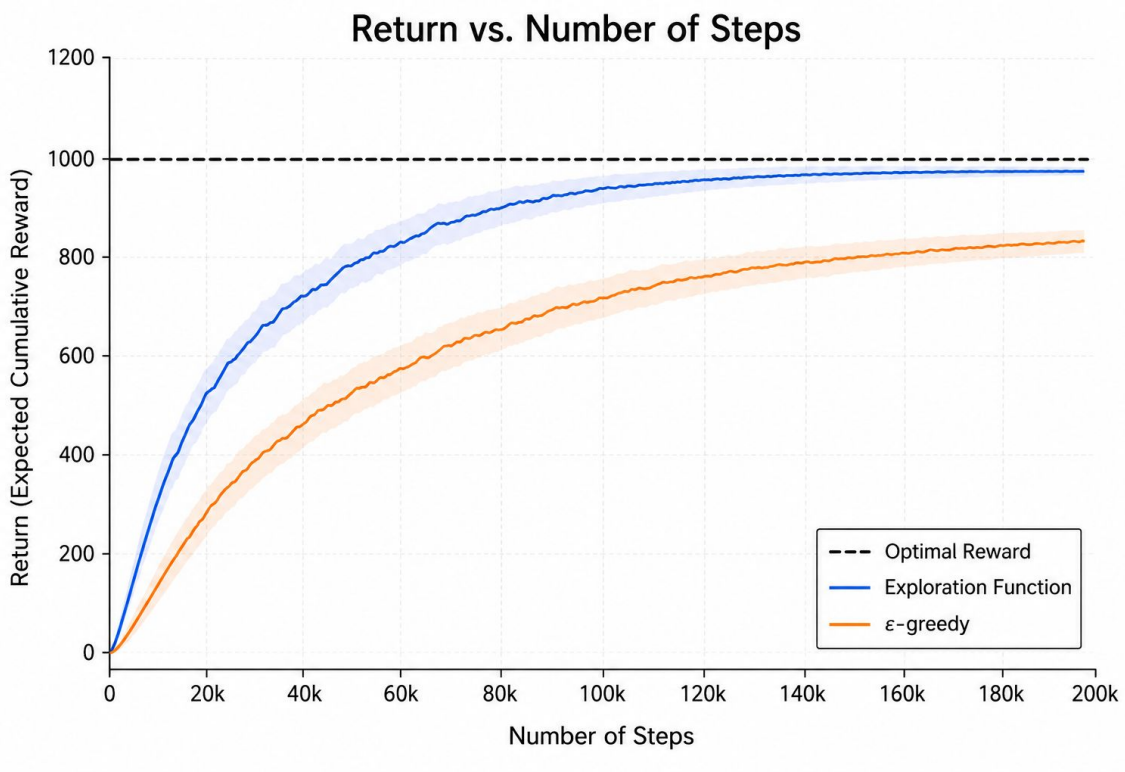


- Even if you learn the optimal policy, you still make mistakes along the way!

$$\text{Regret}(T) = \mathbb{E}\left[\sum_{t=1}^{T} r_t^*\right] - \mathbb{E}\left[\sum_{t=1}^{T} r_t\right]$$

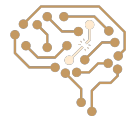
- Minimizing regret goes beyond learning to be optimal – it requires optimally learning to be optimal
- Example: random exploration and exploration functions both end up optimal, but random exploration has higher regret

Regret Example



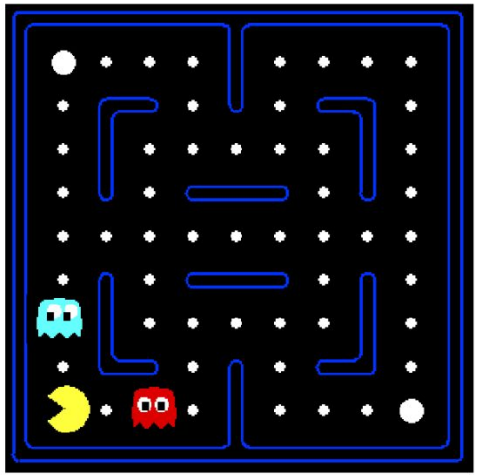
Generalizing Across States

- Basic Q-Learning keeps a table of all q-values
- In realistic situations, we cannot possibly learn about every single state!
  - Too many states to visit them all in training
  - Too many states to hold the q-tables in memory
- Instead, we want to generalize:
  - Learn about some small number of training states from experience
  - Generalize that experience to new, similar situations
  - This is a fundamental idea in machine learning, and we'll see it over and over again

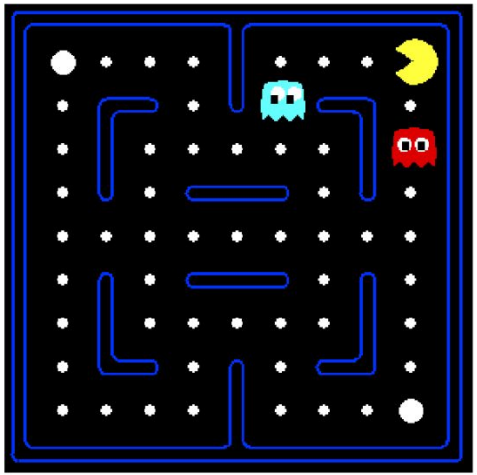


Example: Pacman

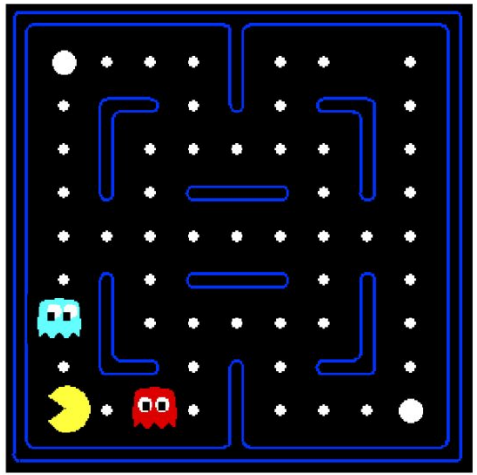
Let's say we discover through experience that this state is bad:



In naïve q-learning, we know nothing about this state:

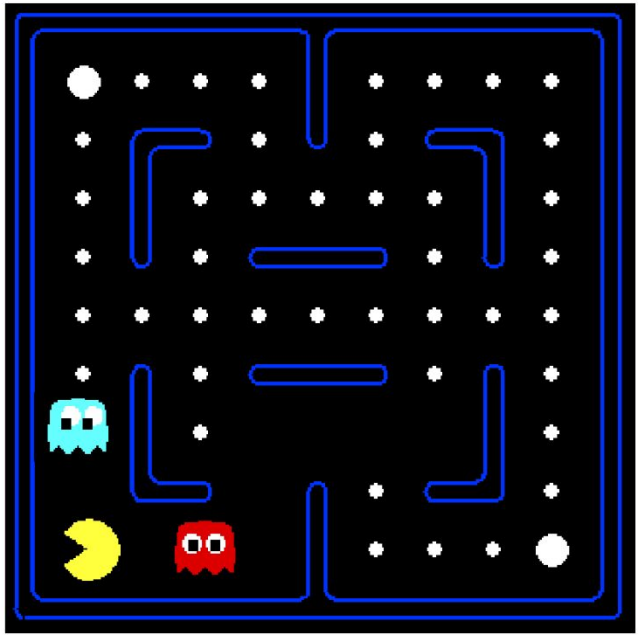


Or even this one!



Example: Pacman

- Solution: describe a state using a vector of features (properties)
  - Features are functions from states to real numbers (often 0/1) that capture important properties of the state
  - Example features:
    - Distance to closest ghost
    - Distance to closest dot
    - Number of ghosts
    - 1 / (dist to dot)^2
    - Is Pacman in a tunnel? (0/1)
    - Is it the exact state on this slide?
    - …… etc.
  - Can also describe a q-state (s, a) with features (e.g. action moves closer to food)



AI Research

- Prerequisites
- Approaches to Join a Research Team/Lab
- Research Fields

Prerequisites

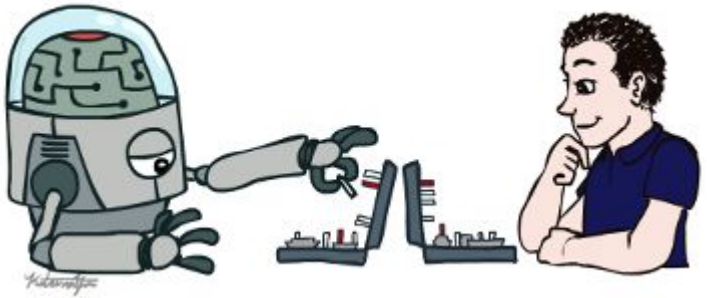
- Deep Learning
  - Stanford: CS230, CS231, …
  - SUT: Deep Learning Course from Dr. Soleymani

- Machine Learning
  - Stanford: cs229
  - SUT: Machine Learning Course from Dr. Rohban and Dr. Soleymani

- Best Time: The summer between semester 4 and 5

Related Courses

- Machine Learning
- Deep Learning
- System-2 AI
- Planning
- Reinforcement learning
- NLP
- Modern Information Retrieval
- Image Processing
- Speech Processing
- …



Approaches to Join a Research Team/Lab

- Direct Approach:
  - Email to the professor
  - Introduce your background and interests
  - Passing prof’s courses and being TA improve your chance

- Indirect Approach:
  - Joining to the project of a master or PhD student
  - Being familiar with lab members improves your change

Research Fields: Data

- Text
- Image
- Audio
- Structured Data (Graph, Table, ...)

Research Fields: Algorithms/Models

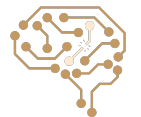
- Transformer
- Diffusion
- CNN
- RNN/LSTM

Research Fields: Philosophy

- Human-like AI (Neuroscience, Brain, ….)
- Embodiment: Robotics, Body, …
- Prediction: Everything is about prediction
  - Next Token Prediction in LLMs
  - Image Classification
- World Modeling: Everything is about predicting the next state
- AI as a Tool vs AI as an agent posses agency

Research Fields: Some Keywords

- Out of distribution (OOD) generalization
- Spurious Correlation (Invariant Learning)
- Compositional Generation/Generalization
- Reasoning
- Faithfulness / Deception
- …



و تمام!